# NET2008 DevOps: Practical Assessment 3

Containerized Infrastructure Pipeline | Docker, Terraform and Ansible | 5% of final grade | 50 marks repository + 50 marks live demo

## The situation

Your team collects web server access logs. Nobody reads them. You build a pipeline that turns a log file into a summary and publishes it on a small web page, **from nothing, with one command**. A Python program summarises the log inside a **Docker** container. **Terraform** creates the network, the storage and the two containers. **Ansible** configures the second container as a web server that publishes the summary. A script, **run.sh**, runs all of it. Everything runs on your own laptop (macOS, Linux, or Windows with the Ubuntu WSL 2 terminal). You decide how to write the code.

## What you submit

A **public GitHub repository named net2008-pa3**. You submit only the repository URL, as text, in **Practical Assessment 3 Submission**. The repository root contains exactly this layout:

```
net2008-pa3/
  README.md  .gitignore  run.sh
  app/      logsum.py  Dockerfile  .dockerignore
  data/     access.log               (a copy of PA3_sample.log)
  infra/    main.tf  variables.tf  outputs.tf  terraform.tfvars
  ansible/  inventory.ini  site.yml  group_vars/all.yml  templates/
```

## Stage A. The program and its container

**app/logsum.py** is started as `python logsum.py INPUT_LOG OUTPUT_JSON`. Each line of the input is one request with six fields separated by spaces: timestamp, client address, method, path, status, bytes.

```
2026-11-02T10:00:00Z 10.0.0.13 GET /login 200 948
2026-11-02T10:00:07Z 10.0.0.11 GET /status 503 2577
2026-11-02T10:01:14Z 10.0.0.12 GET /api/devices 200 5968
2026-11-02T10:01:21Z 10.0.0.13 GET /static/app.css 404 2577
```

- A line is **valid** when it has exactly six fields, the status has exactly three digits and the bytes are digits only. Anything else (and any line that is not empty) is **malformed**. Empty lines are ignored.
- The output is a JSON file with these keys: `total_requests` (valid lines), `malformed`, `status_classes` (the four keys `2xx 3xx 4xx 5xx`, always present, 0 when empty; a valid line with another status such as 100 counts in total_requests only), `top_paths` (at most 3 objects `path` and `count`, most requests first, ties by path A to Z), `total_bytes` and `unique_clients`.
- Exit 0 on success. Missing or unreadable input: a message on standard error that names the file, exit 1, no output file. Wrong number of arguments: a line that starts with `usage:`, exit 2. An empty input is valid.
- Only the Python standard library.

Example: the whole `PA3_sample.log` gives exactly the file `PA3_expected_summary.json` (both on Brightspace):

```
{
  "total_requests": 40,
  "malformed": 3,
  "status_classes": {"2xx": 20, "3xx": 5, "4xx": 5, "5xx": 10},
  "top_paths": [{"path": "/login", "count": 11}, {"path": "/index.html", "count": 8}, {"path": "/api/devices",
  "total_bytes": 174930,
  "unique_clients": 9
}
```

Key order and indentation do not matter. Your file is compared as JSON.

**app/Dockerfile** builds the image `net2008-pa3:1.0`. It must: start from a base image with a **fixed tag** (not `latest`); set a `WORKDIR`; copy only what the program needs; run as a **non-root user**; use the **exec form** of `ENTRYPOINT` so that the two arguments can be passed; and prepare an `/out` folder that the non-root user can write to. Add an `.dockerignore`. Hint: a new named volume copies the owner of the folder in the image.

## Stage B. Terraform (folder infra)

One `terraform apply` creates everything, and `terraform destroy` removes everything. Use these names:

| Resource | Name and rules |
|---|---|
| Network | `pa3-net` |
| Volume | `pa3-out`, mounted at `/out` in both containers |
| Image | `net2008-pa3:1.0`, built **by Terraform** from `app/` (read the Docker provider's image `build` block) |
| App container | `pa3-app`: runs the image once with `data/` mounted read-only at `/data`, arguments `/data/access.log /out/summary.json`. It exits when it is done. Hint: look for `must_run` in the provider documentation. |
| Report container | `pa3-report`: image `debian:bookworm-slim`, command `sleep infinity`, port 80 published on `report_port`. |
| Variables | `report_port` (number, default 8800) and `student_id_last4` (string, no default; the last four digits of your student ID, set in `terraform.tfvars`) |
| Labels | Every container has the label `owner` with the value of `student_id_last4` |
| Outputs | `report_url` (for example http://localhost:8800) and `student_id_last4` |

The code must pass `terraform fmt -check` and `terraform validate`. Do not commit state files or the `.terraform` folder; do commit `.terraform.lock.hcl`.

## Stage C. Ansible (folder ansible)

- Inventory: group `report` with the host `pa3-report`, connected through `community.docker.docker`.
- A bootstrap play installs Python in the container without failing on a second run (the container has none at the start).
- The main play installs nginx, **removes the default site**, and sets nginx up with a **template** so that `GET /summary.json` returns exactly the file the app wrote and `GET /` returns an HTML page. A **handler** reloads nginx when the configuration changes, and nginx is running at the end.
- The page is a Jinja2 template. It shows the text `Student` followed by `student_id_last4`, and the number `total_requests` that Ansible **reads from the summary file** in the container (look at the `slurp` module and the `from_json` filter).
- Every task has a name. Variables live in `group_vars/all.yml`. The second run of the playbook reports `changed=0` for the host.

## Stage D. The pipeline and the repository

- `run.sh` (bash, exit code 0 only if everything worked) runs: `terraform init`, `terraform apply -auto-approve`, then `ansible-playbook` with `student_id_last4` read from the Terraform output (one place for the value), then checks with `curl` that `/summary.json` is valid JSON and prints it.
- It must work from a clean state: after `terraform destroy` on a laptop that has Docker, Terraform and Ansible.

- **README.md**: 15 lines or fewer: what it does, what to install, how to run it. **.gitignore**: state files, `.terraform/`, Python caches.
- At least **8 meaningful commits** over several days, and at least one feature branch merged into `main`. No secrets, no state, no generated files in the repository.

## How this is marked

The repository is worth 50 marks and is checked by running your pipeline on your instructor's computer with a **different input log** in `data/access.log`, so a program tuned to the sample will fail. Stage A alone is worth 16 marks: finish the stages in order, and a pipeline that works up to Stage B still earns most of its marks.

| Stage | What is checked | Marks |
|---|---|---|
| A. Program and container | Correct summary on hidden logs (8), Dockerfile rules (5), runs in the container and writes to the volume (3) | 16 |
| B. Terraform | fmt and validate (2), the five resources with the right names (6), variables, labels and outputs (3), applies from clean and destroys cleanly (3) | 14 |
| C. Ansible | Inventory and bootstrap (2), nginx, template and handler working (4), idempotent second run (3), values read from the summary and Terraform (3) | 12 |
| D. Pipeline and repository | run.sh from clean (3), README and .gitignore (2), commit history and merged branch (2), no state or secrets committed (1) | 8 |
| **Repository total** | | **50** |
| **Live demo** | In the first lab after the deadline | **50** |
| **Total** | | **100** |

**Half of this assessment is a live demo** in your assigned lab session, in the first lab after the deadline. You run your pipeline from a clean state, explain what each stage does and why, and make a small change that your instructor asks for. You must attend your assigned session. If you wrote and understand every line, the demo is easy. Details will be announced later.

## Rules

- Work **alone**. Put your name and student ID in a comment at the top of every file you write. The documentation of Docker, Terraform and Ansible is allowed and expected.
- Deadline: **Saturday, December 5, 11:59 PM**. Do not push after the deadline; your instructor reads the commit history. Late submissions receive 0.
- Commands must work on macOS, Linux and the Ubuntu (WSL 2) terminal. Do not use absolute paths from your own laptop.
- Do not tune your code to the example. Your program is run on input you have not seen.

## Where to start

1) Write `logsum.py` and test it on the sample with `diff` against the expected file. 2) Write the Dockerfile and run it by hand with `docker run -v`. 3) Write the Terraform files, apply, destroy, apply again. 4) Write the Ansible playbook against the running `pa3-report` container. 5) Put it all in `run.sh` and test it from a clean state. Commit after every step.